Robust AI-Image Detection Using a Classical Baseline and a Transfer-Learned CNN

Daniel Harapiak
Department of Computer Science
Western University
London, ON, Canada
dharapia@uwo.ca

Aydan Karmali
Department of Computer Science
Western University
London, ON, Canada
akarma5@uwo.ca

Maximillian Cope
Department of Computer Science
Western University
London, ON, Canada
mcope4@uwo.ca

***Abstract*—This work investigates binary AI-image detection under realistic deployment constraints. The task is to classify natural images (real) versus AI-generated images using a reproducible pipeline that stores run outputs for traceability. A balanced subset from the GenImage benchmark is used, with training/validation/testing split into 22,400/4,800/4,800 images. Two models are compared: (1) a classical handcrafted-feature baseline using logistic regression and (2) a transfer-learned ResNet-18 convolutional neural network (CNN). In clean test conditions, the baseline reaches 0.7142 accuracy and 0.7211 F1, while the CNN reaches 0.9465 accuracy and 0.9452 F1. Robustness tests on JPEG compression and resizing show major degradation for the CNN under aggressive perturbations (e.g., JPEG Q40 F1 = 0.5945), indicating sensitivity to distribution shift despite strong clean performance. The key contribution is a concise and reproducible comparison between classical and deep methods, including robustness analysis and practical limits of generalization. The main conclusion is that strong clean-set performance does not imply robustness under common post-processing, and that artifact-based experiment tracking plus robustness evaluation are essential for reliable reporting; future work will expand generator coverage and robustness-aware training.**

***Index Terms*—AI image detection, deep learning, computer vision, transfer learning, robustness, logistic regression, ResNet**

I. Introduction

Recent text-to-image generators have made synthetic image content widely accessible, increasing the need for reliable AI-image detection systems. Modern generation pipelines span GAN-based methods [1], [2] and diffusion-based methods [3], [4], and image realism has improved rapidly over only a few years. As quality improves, simple visual inspection becomes less reliable and automated forensics-style detection becomes more important for practical use cases such as dataset curation, moderation support, and integrity checks in academic and industrial workflows [5].

At the same time, prior work shows that detector performance is highly sensitive to data distribution, generator family, and post-processing transformations [6], [7]. In other words, a detector can score very well on a clean benchmark split but still degrade significantly when inputs are compressed, resized, or produced by unseen generators. This gap between clean-benchmark performance and deployment reality motivates stronger evaluation design rather than relying on a single headline accuracy number.

This work addresses a practical binary classification problem: detect whether an input image is real (natural) or AI-generated. It includes a classical baseline, a deep CNN model, and robustness testing. The central challenge is not only maximizing clean-set accuracy, but understanding how performance changes under realistic degradations such as compression and resizing.

The implementation emphasizes reproducibility: fixed split manifests, explicit model configurations, stored run artifacts, and evaluation outputs that support independent verification. This design improves methodological transparency and enables consistent comparison between model families. Beyond clean-only model comparison, robustness analysis is integrated into the evaluation to quantify sensitivity to deployment-relevant post-processing. The results are practically relevant for deployment and informative for understanding the gap between benchmark accuracy and performance under distribution shift.

The contribution of this work lies in a reproducible comparison of classical and transfer-learned deep models under both clean and degraded imaging conditions, with traceable outputs that support verification of reported metrics. Rather than proposing a new detection architecture, the study provides an evidence-based evaluation workflow that surfaces robustness and generalization trade-offs relevant to practical deployment.

This work focuses on three research goals. **O1** is to build a reproducible AI-image detection pipeline using fixed train/validation/test splits. **O2** is to compare a classical handcrafted-feature baseline against a transfer-learned CNN under the same split conditions. **O3** is to evaluate robustness of the best model under JPEG compression and resolution reduction.

The report is structured as follows: Section II reviews background and related work; Section III explains objectives and methodology; Section IV presents results and analysis; Section V concludes with limitations, lessons learned, and future work.

II. Background and Related Work

A. AI-image detection and dataset choice

AI-image detection has become an important computer vision problem due to rapid improvements in generative models.

The GenImage benchmark is designed for this task and is widely used for evaluating detector generalization across generators and settings [8]. To keep the project computationally manageable while preserving class balance and reproducibility, this implementation uses two generators (BigGAN and Midjourney) and fixed split manifests. This follows prior evidence that cross-generator setup choices strongly affect reported detector performance [8], [9].

B. Classical versus deep approaches

Classical approaches often rely on handcrafted frequency/statistical cues and linear or kernelized classifiers. They are easier to interpret and cheaper to train, but usually underperform deep models on complex visual distributions.

Deep CNNs, especially transfer-learned ResNet variants, are strong baselines for image classification tasks and are course-aligned in terms of architecture, optimization, and evaluation methodology [10], [11]. Prior fake-image detection work also reports that both semantic and frequency-domain cues can be useful, depending on the generator family and test conditions [6], [12].

C. Robustness and distribution shift in prior work

Prior studies in synthetic-image detection and domain generalization report that detector quality can drop under shifts such as compression, resampling, and generator changes [6], [7], [9]. This motivates a report design that includes both clean test performance and controlled perturbation tests. In this project, JPEG and resize were selected because they are realistic user-side transformations and were explicitly prioritized during project planning.

D. Research gap addressed

Many class projects report only clean accuracy. This work explicitly compares a classical baseline with a transfer-learned CNN and then evaluates robustness under deployment-relevant degradations (JPEG and resize), making results more informative than clean-only reporting.

III. Methods

A. Research objectives and significance

O1 provides reproducibility via fixed split manifests and saved run artifacts.
O2 provides a fair model comparison between classical and deep learning strategies.
O3 provides practical insight into deployment risks under image quality shifts.

These objectives are significant because they balance theoretical model comparison with practical detector behavior under noisy real-world conditions.

B. Dataset and splits

The project uses a GenImage-derived subset with two generators, BigGAN and Midjourney, and two classes, real (nature) and ai. The combined split counts are 22,400 for training, 4,800 for validation, and 4,800 for testing.

Dataset splits are defined using fixed manifest tables and consumed by a dataset loader that performs on-the-fly image loading from file paths with associated binary labels. All major runs store the resolved configuration and output artifacts to timestamped run folders for traceability.

TABLE I
Per-Generator Split Counts (per class)

Split	BigGAN (ai/real)	Midjourney (ai/real)	Combined
Train	5600 / 5600	5600 / 5600	22400
Val	1200 / 1200	1200 / 1200	4800
Test	1200 / 1200	1200 / 1200	4800

C. Research question and hypothesis

The guiding research question is: *How much does a transfer-learned CNN improve AI-image detection performance over a classical handcrafted-feature baseline, and how stable is that improvement under common degradations?*

Hypothesis H1: the CNN will significantly outperform the baseline on clean test data.
Hypothesis H2: both models, especially the CNN, will lose performance under stronger JPEG and resize perturbations.

From a systems perspective, this question matters because it supports a defensible comparison between model families and because robustness under common transformations is often as important as clean-set performance for practical use.

D. Model 1: Classical baseline

The baseline extracts handcrafted image features and trains a logistic regression classifier with standardization. The feature set includes a grayscale histogram (32 bins), RGB channel means and standard deviations, gradient magnitude statistics, FFT-derived low/high-frequency energy statistics, and an entropy feature.

The classifier is implemented as `StandardScaler + LogisticRegression (lbfgs)`. In the baseline configuration, logistic regression used `C = 1.0` and `max_iter = 3000`, with predictions generated using a fixed decision threshold of `0.5`.

E. Model 2: Transfer-learned CNN

The deep model is a transfer-learned ResNet-18 binary classifier [10]. The final fully-connected layer is replaced with one output logit and trained using BCEWithLogitsLoss. The main settings are image size 224×224, 8 epochs, batch size 48, learning rate 3×10^{-4}, weight decay 1×10^{-5}, and decision threshold 0.5.

Training uses `build_image_transform(is_train=true)`, which resizes to 224 and applies a random horizontal flip with probability 0.5; both training and evaluation normalize inputs using mean/std `[0.485, 0.456, 0.406]` and `[0.229, 0.224, 0.225]`. Validation F1 is used for best-checkpoint selection.

F. Implementation strategy

The project follows a cyclic build/test/feedback process: fixed split manifests are finalized and converted to split CSV files; the classical baseline is trained and evaluated as a comparison anchor; the CNN is then trained and the best checkpoint is selected using validation F1; robustness evaluation is run on clean, JPEG, and resize variants; and finally detector inference is run on unseen user-provided images. Each stage writes reproducibility artifacts (`config_used.yaml`, `metrics.json`, summaries, plots, and prediction outputs) to timestamped run folders.

This implementation strategy also reduces reporting ambiguity: reported metrics and figures can be traced back to stored run outputs, supporting reproducibility and independent verification.

Experiments were executed as separate training, evaluation, and inference stages driven by configuration files; each stage creates a timestamped run directory and saves the resolved configuration alongside per-condition metrics, plots (e.g., confusion matrices and ROC curves), per-image prediction outputs, and representative misclassification examples.

Software-wise, the pipeline is implemented in Python with scikit-learn for the logistic-regression baseline and PyTorch/torchvision for the ResNet-18 CNN; the compute device is selected automatically at runtime based on CUDA availability.

G. Evaluation metrics and robustness protocol

For both models, the report uses:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN} \tag{1}$$

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{2}$$

Also reported are accuracy, confusion matrix, ROC-AUC, and per-image predictions. ROC-AUC is computed only when the evaluation set contains both classes. Robustness is evaluated on CNN outputs using JPEG qualities Q85, Q60, and Q40, and resize scales of 0.75 and 0.5 (downsample then upsample).

Descriptive statistics are reported directly from confusion matrices and aggregated metrics (precision, recall, F1, ROC-AUC) for each condition. Data gathering instruments are the saved split CSV manifests and run artifacts generated by training/evaluation scripts. The baseline classifier is a regularized logistic regression pipeline [13], while ROC-AUC interpretation follows standard binary-classification practice [14].

H. Threats to validity

The system is trained on two generators only (BigGAN and Midjourney), so out-of-distribution generalization to newer models (e.g., ChatGPT/Gemini image outputs) is expected to be weaker. Additional threats include limited hyperparameter search and single-project compute constraints.

Another validity threat is that resizing all images to a common input size can suppress fine-grained generator-specific artifacts. While this standardization is necessary for stable CNN training and fair baseline comparison, it may hide subtle cues that exist at native resolution.

IV. Results and Analysis

A. Clean model comparison

Table II summarizes test metrics for baseline and CNN.

Reported tables and figures are generated directly from stored evaluation outputs produced by the pipeline, including aggregated metrics, confusion matrices, ROC curves (when defined), and per-image prediction records. This artifact-based workflow enables traceability from the plotted results back to the underlying metrics and predictions, supporting reproducible validation of the reported numbers.

TABLE II
Clean Test-Set Comparison

Model	Acc.	Prec.	Recall	F1	ROC-AUC
Baseline	0.7142	0.7040	0.7392	0.7211	0.7951
CNN (ResNet-18)	0.9465	0.9685	0.9229	0.9452	0.9887

The CNN substantially outperforms the classical baseline on all clean-set metrics, supporting the expected advantage of transfer-learned deep features for this task.

Figure 1 and Figure 2 provide baseline decision behavior on the test set. Compared to the CNN, the baseline has both lower separability and higher class confusion, which is consistent with the large metric gap in Table II. The consolidated visual comparison in Figure 3 presents the same clean-condition ranking in a compact report-ready format.

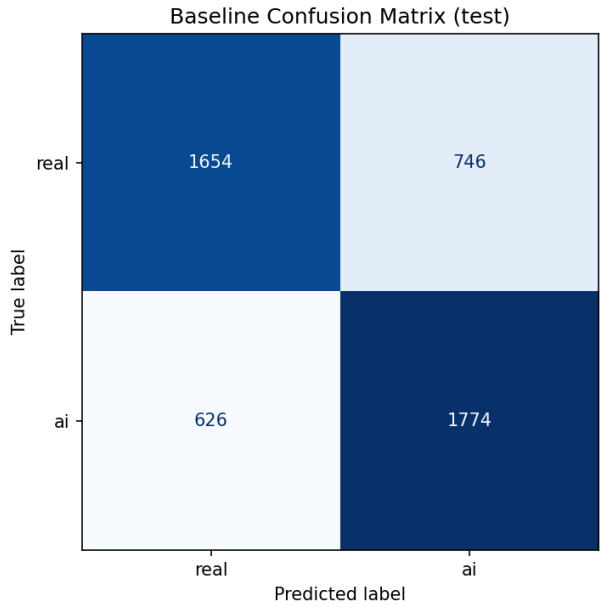

Fig. 1. Baseline confusion matrix on clean test set.

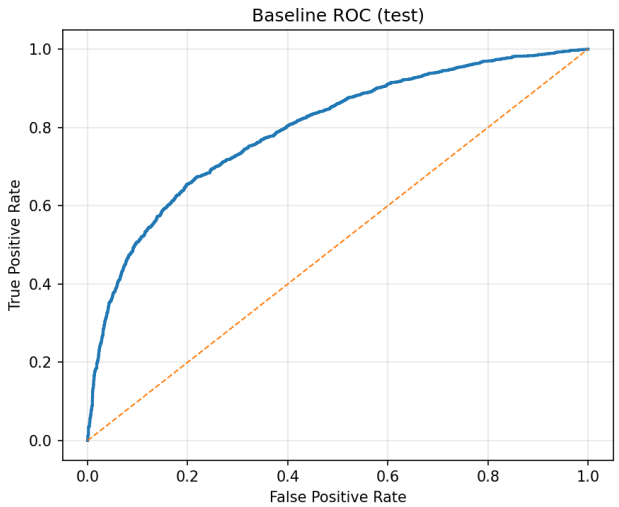

Fig. 2. Baseline ROC curve on clean test set.

Clean Model Comparison (Test Set)

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Baseline test	0.7142	0.7040	0.7392	0.7211	0.7951
CNN test	0.9465	0.9685	0.9229	0.9452	0.9887

Fig. 3. Clean model comparison table exported from run artifacts.

B. CNN training dynamics

Training curves indicate stable optimization and convergence within 8 epochs.

The training history indicates that validation performance improves quickly and then plateaus, suggesting the chosen epoch budget is sufficient for this project scope. This matters methodologically because it reduces the chance that the CNN advantage is simply due to under-training of one model family. This trend is visible in Figure 4, where validation metrics stabilize in later epochs.

C. Confusion matrix and ROC interpretation

On clean data, the CNN confusion matrix shows low error in both classes, and ROC performance is near ideal.

In contrast, the baseline confusion matrix shows more symmetric class overlap, which is expected when handcrafted features do not fully capture generator artifacts across diverse content. ROC separation is therefore weaker for the baseline, matching its lower AUC. For the CNN, Figure 5 and Figure 6 show strong class separation and high ranking quality on the clean test set.

D. Robustness results

Table III shows robustness outcomes for the CNN.

The model remains fairly strong under mild resizing and JPEG Q85, but performance drops sharply under heavier JPEG compression (especially recall) and more aggressive

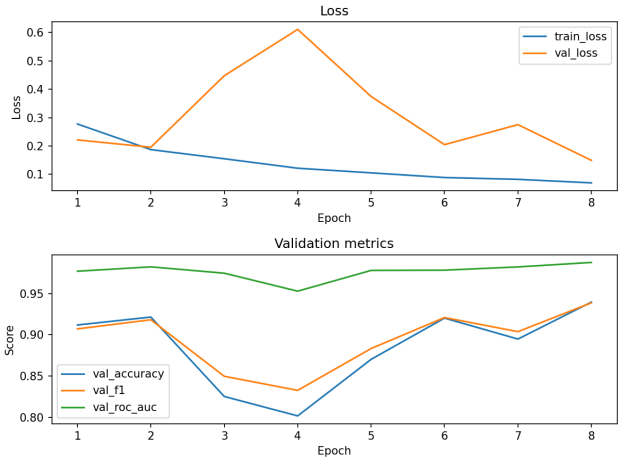

Fig. 4. CNN training and validation history across epochs.

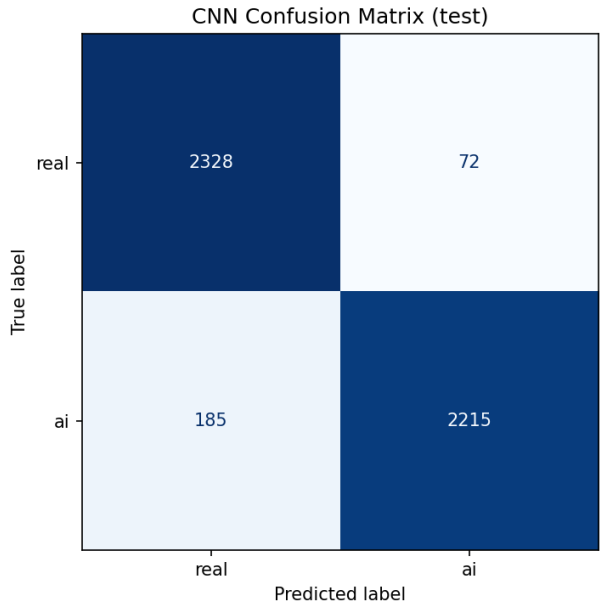

Fig. 5. CNN confusion matrix on clean test set.

TABLE III
CNN ROBUSTNESS ACROSS DEGRADATIONS

Condition	Accuracy	F1	ROC-AUC
Clean	0.9465	0.9452	0.9887
JPEG Q85	0.7931	0.7501	0.9159
JPEG Q60	0.7388	0.6596	0.8830
JPEG Q40	0.7050	0.5945	0.8728
Resize 0.75	0.9006	0.9024	0.9619
Resize 0.5	0.7521	0.7763	0.8209

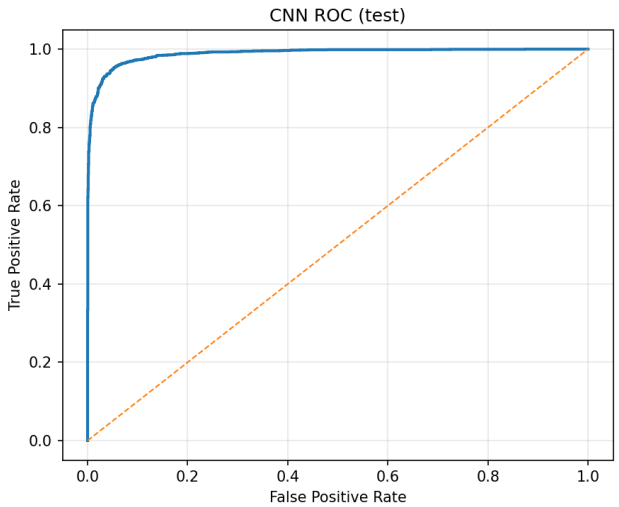

Fig. 6. CNN ROC curve on clean test set.

downscaling. This indicates the detector is learning useful but somewhat fragile visual cues.

These conditions correspond to the evaluation tags generated from `configs/eval_config.yaml` ("clean", "jpeg_q85", "jpeg_q60", "jpeg_q40", "resize_0p75", and "resize_0p5"), each producing a separate confusion matrix and ROC curve.

From the confusion matrices, the dominant failure mode under JPEG Q40 is an increase in false negatives for the AI class (AI images mislabeled as real). This aligns with the recall collapse from 0.9229 (clean) to 0.4325 (JPEG Q40), while precision stays high due to more conservative positive predictions.

This precision-recall imbalance is practically important. In a deployment scenario, high false negatives for AI images can be more harmful than moderate false positives because synthetic content may bypass screening. Therefore, threshold tuning and robustness-aware calibration should be considered part of future deployment preparation, not optional post-processing. Figure 7 summarizes robustness metrics across all tested perturbations, while Figure 8 and Figure 9 provide the clean-vs-degraded confusion-matrix contrast discussed above.

Evaluate Metrics Summary

Condition	Accuracy	Precision	Recall	F1	ROC-AUC
clean	0.9465	0.9685	0.9229	0.9452	0.9887
jpeg_q85	0.7931	0.9472	0.6208	0.7501	0.9159
jpeg_q60	0.7388	0.9463	0.5062	0.6596	0.8830
jpeg_q40	0.7050	0.9505	0.4325	0.5945	0.8728
resize_0p75	0.9006	0.8863	0.9192	0.9024	0.9619
resize_0p5	0.7521	0.7072	0.8604	0.7763	0.8209

Fig. 7. Evaluation summary table across clean and degraded conditions.

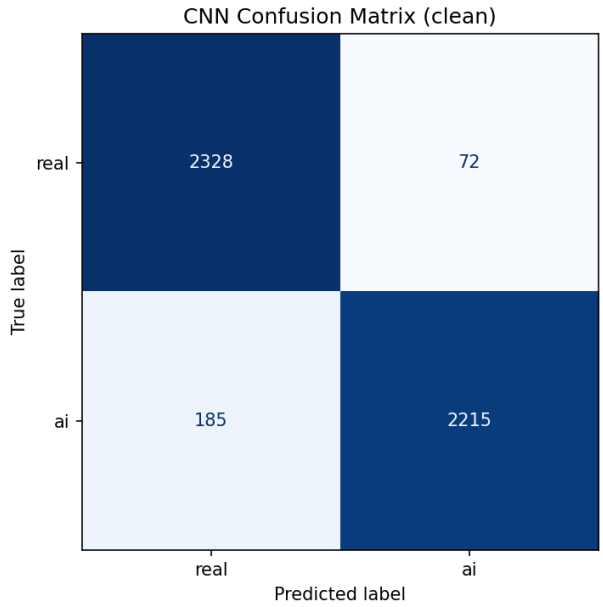

Fig. 8. CNN confusion matrix under clean condition during robustness evaluation.

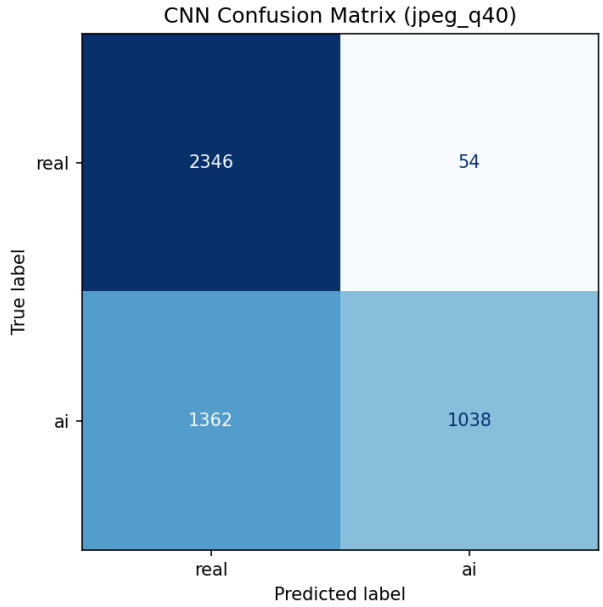

Fig. 9. CNN confusion matrix under heavy JPEG compression (Q40).

E. Discussion

H1 is strongly supported: the transfer-learned CNN significantly outperforms the classical baseline in clean conditions. H2 is also supported: robustness testing reveals that strong clean metrics do not guarantee stable behavior under common image degradations.

In terms of practical impact, the system is effective under the evaluated benchmark conditions, but should not be presented as universally robust AI-image detection. Since training used only BigGAN and Midjourney, performance is expected to drop for unseen modern generators (e.g., ChatGPT/Gemini outputs). This limitation is central to interpreting the results.

Taken together, the results demonstrate progress not only in predictive performance on the clean benchmark split, but also in establishing a traceable evaluation process for diagnosing failure modes under realistic perturbations.

This behavior is consistent with broader findings in the literature, where cross-generator generalization and post-processing robustness remain unresolved even for strong detectors [7]–[9].

V. Conclusion and Future Work

This work delivered a reproducible AI-image detector with an experimental structure spanning a classical baseline, a deep CNN, and robustness evaluation. The CNN achieved high clean performance (accuracy 0.9465, F1 0.9452), while the baseline provided a useful lower-bound reference (accuracy 0.7142, F1 0.7211). Robustness experiments showed substantial sensitivity to compression and aggressive resizing, which is the most important practical finding.

Future work includes training with additional generators to improve out-of-distribution generalization, augmentation and robustness-aware training for compression and resize resistance, threshold calibration with error analysis by source generator or domain, and testing lightweight ensemble or frequency-aware hybrid models.

Main lessons learned are that reproducible artifacts and robust evaluation are essential, and that clean-test performance alone can overestimate real-world reliability.

Acknowledgment

The author acknowledges use of the GenImage dataset and open-source PyTorch tooling.

References

[1] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2014. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2014/hash/f033ed80deb0234979a61f95710dbe25-Abstract.html

[2] T. Karras, S. Laine, and T. Aila, "Analyzing and improving the image quality of stylegan," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020. [Online]. Available: https://doi.org/10.1109/CVPR42600.2020.00853

[3] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/hash/4c5bcfec8584af0d967f1ab10179ca4b-Abstract.html

[4] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. [Online]. Available: https://doi.org/10.1109/CVPR52688.2022.01042

[5] L. Verdoliva, "Media forensics and deepfakes: An overview," *IEEE Journal of Selected Topics in Signal Processing*, vol. 14, no. 5, pp. 910–932, 2020. [Online]. Available: https://doi.org/10.1109/JSTSP.2020.3002101

[6] S.-Y. Wang, O. Wang, R. Zhang, A. Owens, and A. A. Efros, "Cnn-generated images are surprisingly easy to spot... for now," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020. [Online]. Available: https://arxiv.org/abs/1912.11035

[7] L. Chai, D. Bau, S. Lim, and P. Isola, "What makes fake images detectable? understanding properties that generalize," in *European Conference on Computer Vision (ECCV)*, 2020. [Online]. Available: https://arxiv.org/abs/2008.10588

[8] "Genimage: A million-scale benchmark for detecting ai-generated image," https://arxiv.org/abs/2306.08571, 2023, accessed: 2026-03-26.

[9] "Towards universal fake image detectors that generalize across generative models," https://arxiv.org/abs/2302.10174, 2023, accessed: 2026-03-26.

[10] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016. [Online]. Available: https://doi.org/10.1109/CVPR.2016.90

[11] PyTorch Contributors, "Torchvision models documentation," https://pytorch.org/vision/stable/models.html, accessed: 2026-03-26.

[12] "Leveraging frequency analysis for deep fake image recognition," https://arxiv.org/abs/2003.08685, 2020, accessed: 2026-03-26.

[13] scikit-learn Developers, "scikit-learn logisticregression documentation," https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html, accessed: 2026-03-26.

[14] T. Fawcett, "An introduction to roc analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006. [Online]. Available: https://doi.org/10.1016/j.patrec.2005.10.010